Computer Graphics

Professor YoungWoon Cha

Computer Science and Engineering
Metaverse Convergence
Graduate School

Contents

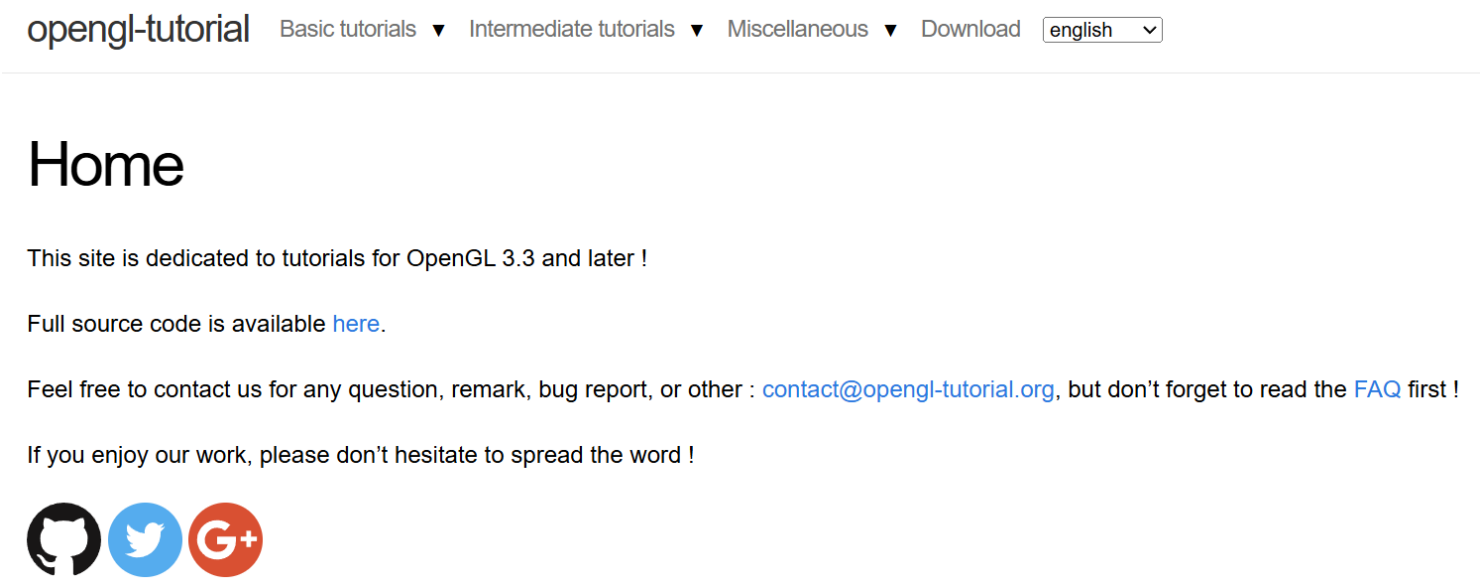
- OpenGL Tutorial II
- Matrices
- Textured Cube
- Keyboard and Mouse
- HW2 Instructions

KU KONKUK UNIVERSITY

OpenGL Tutorial II

OpenGL Tutorial

- In this lab, we will go through a series of OpenGL tutorials
  - Reference: https://www.opengl-tutorial.org/
  - The source code is available on this website



KU KONKUK UNIVERSITY

Matrices

Tutorial 3: Matrices

- Matrix representation
  - with C++ GLM library

Let's now see what happens to a vector that represents a direction towards the -z axis: (0,0,-1,0)

$$\begin{bmatrix} 1 & 0 & 0 & 10 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 0 \\ 0 \\ -1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1*0 & + & 0*0 & + & 0*0 & + & 10*0 \\ 0*0 & + & 1*0 & + & 0*0 & + & 0*0 \\ 0*-1 & + & 0*-1 & + & 1*-1 & + & 0*-1 \\ 0*0 & + & 0*0 & + & 0*0 & + & 1*0 \end{bmatrix} = \begin{bmatrix} 0+0+0+0 \\ 0+0+0+0 \\ 0+0+-1+0 \\ 0+0+0+0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -1 \\ 0 \end{bmatrix}$$

…i.e. our original (0,0,-1,0) direction, which is great because, as I said ealier, moving a direction does not make sense.

So, how does this translate to code?

In C++, with GLM:

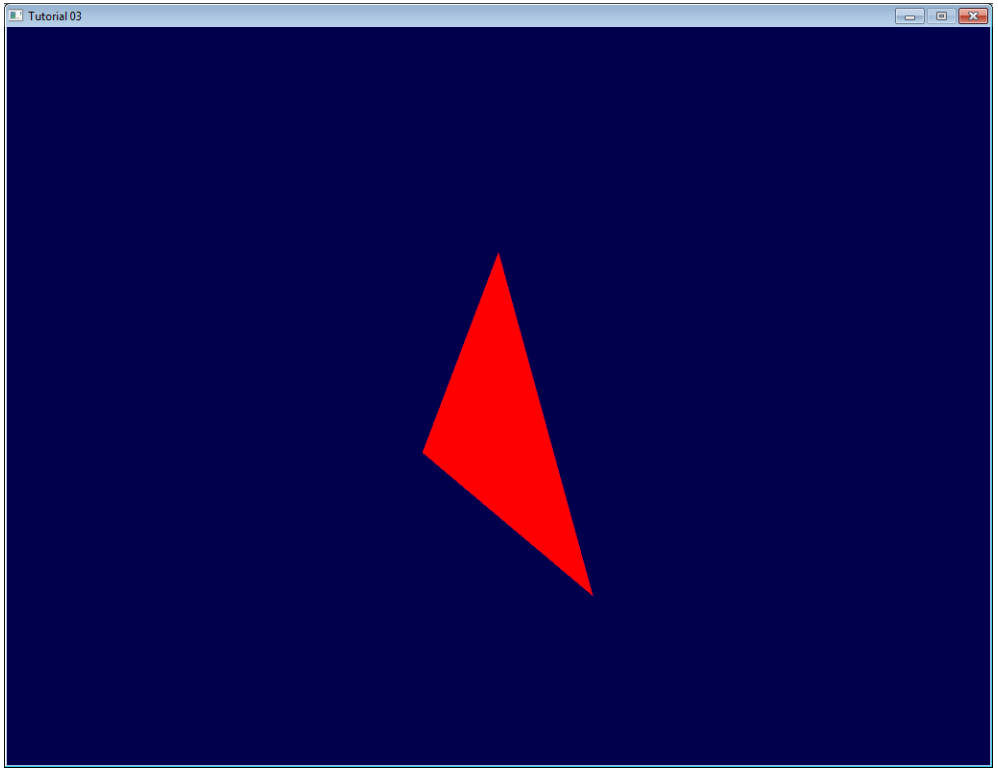
```
#include <glm/gtx/transform.hpp> // after <glm/glm.hpp>

glm::mat4 myMatrix = glm::translate(glm::mat4(), glm::vec3(10.0f, 0.0f, 0.0f));
glm::vec4 myVector(10.0f, 10.0f, 10.0f, 0.0f);
glm::vec4 transformedVector = myMatrix * myVector; // guess the result
```

Tutorial 3: Matrices

- Transformations
  - Model, View, Projection Matrices

in C++, with GLM:

```
glm::mat4 myModelMatrix = myTranslationMatrix * myRotationMatrix * myScaleMatrix;
glm::vec4 myTransformedVector = myModelMatrix * myOriginalVector;
```

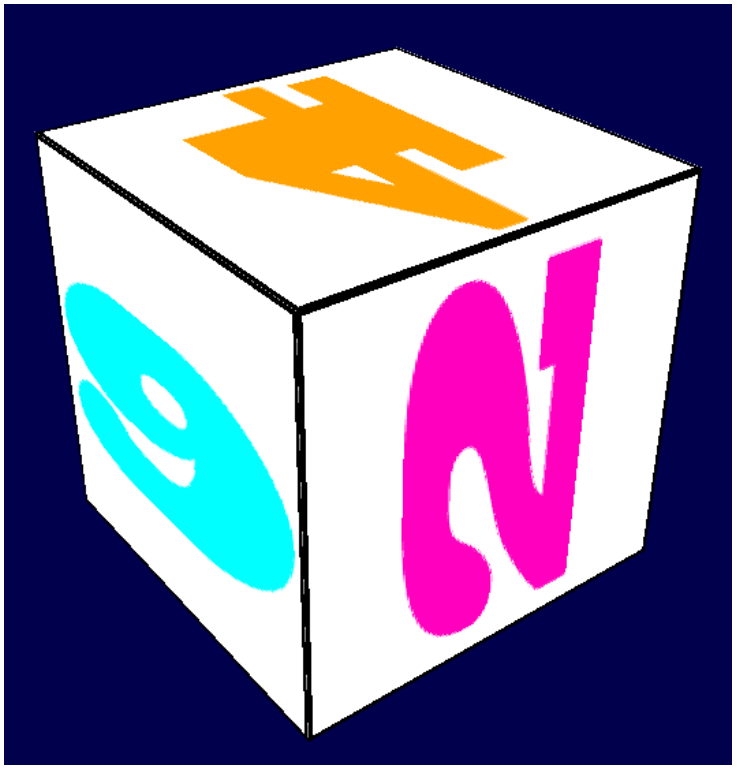


KU KONKUK UNIVERSITY

Textured Cube

Tutorial 5: Textured Cube

- UV coordinates
- Using Textures

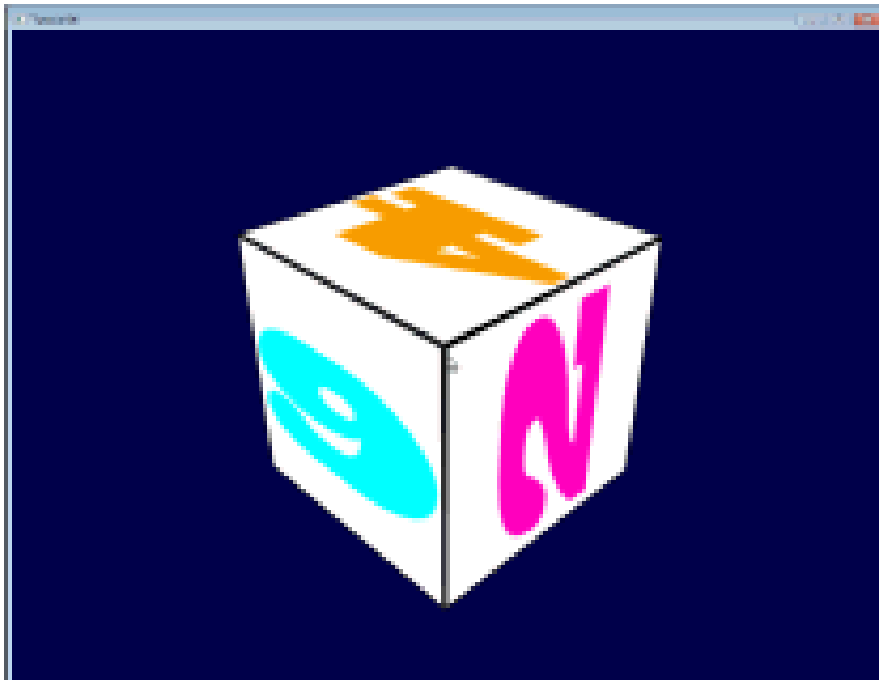


KU KONKUK UNIVERSITY

Keyboard and Mouse

Tutorial 6: Keyboard and Mouse

- Implementing Scene Rotation using Mouse
- Implementing Scene Translation using Keyboard



KU KONKUK UNIVERSITY

HW2 Instructions

HW2: Question 1

- (Required) Draw at least two differently textured cubes
  - Ensure the cubes have **different sizes** and use **different texture images**

- (Required) Keyboard + Mouse Interface
  - Users must be able to rotate and translate the scene using a keyboard or mouse

- (Required) Implement and demonstrate a new feature nicely
  - e.g., Drawing differently shaped objects (other than cubes)
  - e.g., Different user interfaces, such as FPS-style mouse controls for walking

Submission Guidelines

- Important Notes
  - Your code must include a single project for this question
    - Please do not submit any tutorials.
    - Submit only the code related to this question

- Submission Requirements
  - Presentation video: Under 5 minutes and 30MB maximum (MP4 format)
  - Presentation slides: PDF format
  - Source code: VS 2022 project (ZIP file)

Summary

- OpenGL Tutorial II
- Matrices
- Textured Cube
- Keyboard and Mouse
- HW2 Instructions

End of Class

Thank you

E-mail: youngcha@konkuk.ac.kr